Sparse Autoencoders & Mechanistic Interpretability

Cunningham et al. (2023)

Ruiting Jiang

Department of Philosophy, University of Notre Dame

AI & Philosophy Reading Group

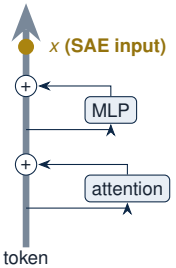
Representation of Tokens

At each token position, the model has an internal representation:

$$x = (x_1, \ldots, x_d) \in \mathbb{R}^d.$$

Where does x come from? Each token carries a vector up through the layers, called the **residual stream**. Every layer reads this vector, computes something, and *adds* its result to x. So the vector is like the token's working memory, which every layer reads from and writes to.

The SAE is trained on this vector, taken after a particular layer.



Question

Can each x be explained by using a small number of features?

Unknown Features vs. Learned Features

The paper distinguishes

$$\underbrace{g_1, \ldots, g_m}_{\text{unknown “true” network features}}$$

from

$$\underbrace{f_1, \ldots, f_n}_{\text{learned dictionary features}}$$

The hypothesis is

$$x \approx \sum_j a_j g_j$$

with only a few nonzero a_j.

Goal

The SAE tries to learn a dictionary such that

$$\forall g_j \qquad \exists f_k \qquad g_j \approx f_k.$$

so

$$x \approx \sum_k c_k f_k$$

with only a few nonzero c_k.

It therefore transforms a dense-looking activation

$$x \in \mathbb{R}^d$$

into a sparse feature-coordinate vector

$$c = (c_1, \ldots, c_n), \qquad n > d.$$

The Sparse Autoencoder



The encoder:

$$\boxed{c = \mathrm{ReLU}(Mx + b)}$$

The decoder:

$$\boxed{\hat{x} = M^\top c.}$$

If the i-th row of M is $f_i^\top$, then

$$\hat{x} = \sum_i c_i f_i.$$

Why Use M and $M^\top$?

Suppose f_i formed an orthonormal basis and

$$x = \sum_i a_i f_i.$$

Let M be the matrix whose rows are $f_i^\top$, so that $x = M^\top a$. Then we have $a = Mx$.

This motivates using the same matrix M for both the encoder and decoder.

The authors also note that tied weights halve memory use.

The Role of ReLU and Bias

Let f_i be one of the features.
We want to measure the activation of the feature f_i in the input x:

$$\langle f_i, x \rangle.$$

But because of superposition, a weak positive value might arise merely from interference.
Therefore, in the encoder:

$$c_i = \mathrm{ReLU}\big(\langle f_i, x \rangle + b_i\big).$$

If $b_i < 0$, the bias acts like a threshold to ignore interference from other features.

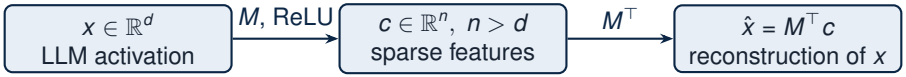
Toy Models of Superposition vs. Sparse Autoencoders

Toy model of superposition (Elhage et al., 2022)



Input = **ground truth**, one coordinate per feature. Squeeze into $m < n$ dims; goal: $\hat{a} \approx a$.

Sparse autoencoder (Cunningham et al., 2023)



Input = **representation learned by the LLM**. Recovers sparse features, then reconstructs x.

Almost an inverse

Toy: $h = \sum_j a_j W_{:,j}$; SAE hypothesis: $x \approx \sum_j a_j g_j$. So x plays the role of the toy model's *hidden* h, and c that of its *input* a, now with no ground truth to check against.

The Loss

SAE minimizes

$$\mathcal{L}(x) = \underbrace{\|x - \hat{x}\|_2^2}_{\text{reconstruction}} + \alpha \underbrace{\|c\|_1}_{\text{sparsity}}$$

with

$$c = \mathrm{ReLU}(Mx + b), \qquad \hat{x} = M^\top c.$$

Each row f_i of M is fixed to length 1, so the penalty cannot be avoided by making f_i longer and c_i smaller.

Two competing goals

- **Reconstruction:**

$$\hat{x} \approx x.$$

- **Sparsity:** use relatively few active features.

Why Does L_1 Encourage Sparsity?

After all,

$$\|(5,0,0,0,0)\|_1 = \|(1,1,1,1,1)\|_1 = 5.$$

So the L_1 norm itself doesn't prefer the sparse vector. Why, then, does L_1 regularization produce sparse solutions?

Acknowledgement

The following examples and explanation were given to me in a lecture by Prof. Xiangliang Zhang, as a comment on and addition to my presentation.

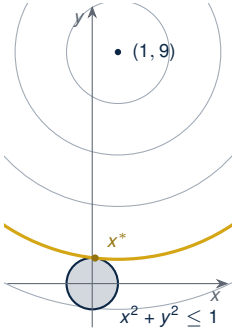
The Optimization Problem

Consider a constrained optimization problem:

$$\min_{x,y} \; f(x,y) = (x-1)^2 + (y-9)^2 \quad \text{s.t.} \quad x^2 + y^2 \le 1,$$

i.e. minimize $x^2 + y^2 - 2x - 18y + 82$ over the unit disk.

- Level sets of f: circles centered at $(1, 9)$.
- Shrink the circles: the optimum x^* is where one *first touches* the disk.



Fact

Let f be strictly convex and C closed and convex. If the unconstrained minimizer of f is not in C, then $\arg\min_C f$ lies on the boundary ∂C.

The Lagrange Multiplier

Write $g(x, y) = x^2 + y^2 - 1 \leq 0$ and form the Lagrangian

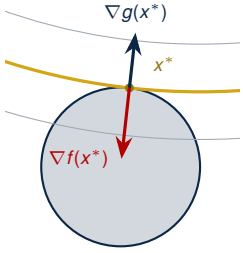
$$\mathcal{L}(x, y, \mu) = f(x, y) + \mu\, g(x, y), \qquad \mu \geq 0.$$

KKT conditions (necessary and sufficient here: the problem is convex and $(0, 0)$ is strictly feasible):

$$\nabla f + \mu \nabla g = 0, \quad g \leq 0, \quad \mu \geq 0, \quad \mu\, g = 0.$$

From $\nabla f + \mu \nabla g = 0$: $2(x - 1) + 2\mu x = 0$,
$2(y - 9) + 2\mu y = 0$, so $(x, y) = \frac{1}{1+\mu}(1, 9)$.

- $\mu = 0$ gives $(1, 9)$: infeasible. So $\mu > 0$, hence $g = 0$: **the constraint is active**.
- $\frac{82}{(1+\mu)^2} = 1 \Rightarrow \mu = \sqrt{82} - 1 \approx 8.06$,
  $x^* = \frac{1}{\sqrt{82}}(1, 9) \approx (0.11,\ 0.99)$.



$\nabla f(x^*) = -\mu\, \nabla g(x^*)$:
level set tangent to the circle

SAE Loss as a Lagrangian

Recall that the SAE is trained with the loss

$$\mathcal{L}_{\text{SAE}} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{reconstruction error}} + \alpha \underbrace{\|c\|_1}_{\text{sparsity penalty}} .$$

Now consider instead a **constrained** problem, of the same shape as the previous example:

$$\boxed{\begin{aligned} \min \quad & f(x, y) \\ \text{subject to} \quad & x^2 + y^2 \leq 1 \end{aligned}} \quad \text{Previous example}$$

$$\longleftrightarrow$$

$$\boxed{\begin{aligned} \min \quad & \|x - \hat{x}\|_2^2 \\ \text{subject to} \quad & \|c\|_1 \leq t \end{aligned}} \quad \text{SAE}$$

Its Lagrangian is

$$\|x - \hat{x}\|_2^2 + \alpha\big(\|c\|_1 - t\big), \qquad \alpha \geq 0.$$

For fixed α, $-\alpha t$ is a constant, so minimizing this Lagrangian is **exactly the same** as minimizing $\mathcal{L}_{\text{SAE}}$: the L_1 regularization can be understood as corresponding to a constraint on how large the total absolute feature activation is.

Same Solution (One Direction)

Simplification. Fix the dictionary $f_1, \ldots, f_n$ and treat c as a free variable, so the reconstruction is

$$\hat{x} = \sum_i c_i f_i.$$

Claim

If c^* minimizes the SAE loss $\|x - \hat{x}\|_2^2 + \alpha\|c\|_1$, then c^* also solves the constrained problem $\min_c \|x - \hat{x}\|_2^2$ s.t. $\|c\|_1 \leq t$, with $t := \|c^*\|_1$.

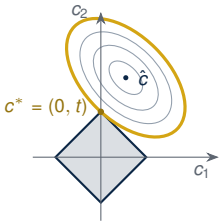
Proof. Write $\hat{x}^* = \sum_i c_i^* f_i$ for the reconstruction from c^*. Let c satisfy $\|c\|_1 \leq t$. Since c^* minimizes the SAE loss,

$$\|x - \hat{x}\|_2^2 \;\geq\; \|x - \hat{x}^*\|_2^2 + \alpha\big(\|c^*\|_1 - \|c\|_1\big) \;\geq\; \|x - \hat{x}^*\|_2^2. \qquad \square$$

So the feasible set is the L_1 diamond $\{\|c\|_1 \leq t\}$, and c^* lies on its boundary $\|c\|_1 = t$, just as in the previous example.

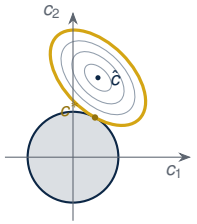
L_1 Constraint Has Diamond Shape

Take $n = 2$ features, $c = (c_1, c_2)$. The level sets of the reconstruction error $E(c) = \|x - c_1 f_1 - c_2 f_2\|_2^2$ are ellipses around $\hat{c}$, the best unconstrained optimal. Grow them until they first touch the constraint set $\{\|c\|_1 \leq t\}$.



L_1: $|c_1| + |c_2| \leq t$

often first touches at a **corner**: $c_1^* = 0$ (**sparse**)



L_2: $c_1^2 + c_2^2 \leq t^2$

first contact at a smooth point: both $c_i^* \neq 0$

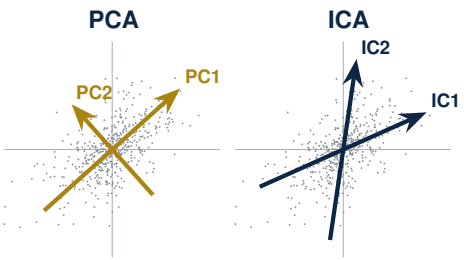
The corners of the diamond lie on the axes, e.g. $(t, 0)$ and $(0, t)$, so a solution at a corner has some coordinates **exactly zero**. In higher dimensions, its edges also lie where some coordinates are 0. Thus, the norm alone does not prefer sparse vectors; sparsity comes from this geometry.

How to Measure Interpretability?

For each learned feature:

1. Find text samples where the learned feature activates.
2. Show examples and activation strengths to an LLM.
3. Ask the LLM for a human-readable hypothesis.
4. Ask another LLM to predict activation on new examples using only that hypothesis.
5. Compare predicted activation with actual activation.

PCA vs. ICA



The same data in both plots: points spread along two “arms”.

- **PCA** finds the directions in which the data spreads out the most, and they must be orthogonal. PC1 points *between* the two arms, so it matches neither.
- **ICA** finds directions that behave like separate, independent sources. They need not be at right angles, so IC1 and IC2 *follow* the arms.
- **SAE** works like ICA: it writes each activation x as a mix of feature directions, $x \approx \sum_k c_k f_k$, where only a few c_k are nonzero, and the directions need not be orthogonal. But an SAE can use *more directions than dimensions*, which superposition requires.

So ICA is the closest baseline to the SAE.

Acknowledgement: this explanation was given to me by Prof. Xiangliang Zhang, as a comment on and addition to my presentation.

Result: SAE Features Are More Interpretable

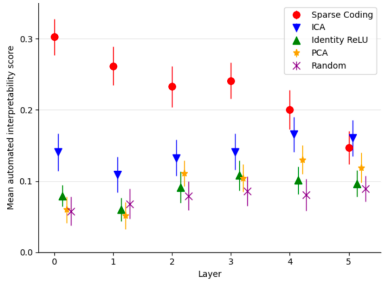


Figure 2: Average top-and-random autointerpretability score of our learned directions in the residual stream, compared to a number of baselines, using 150 features each. Error bars show 95% confidence intervals around means. The feature dictionaries used here were trained for 10 epochs using $\alpha = .00086$ and $R = 2$.

Earlier Layers:

SAE $>$ ICA $>$ other baselines

- **ICA** does relatively well because it looks for statistically independent components.
- **PCA** looks for directions that explain variance, not necessarily directions with one semantic meaning.
- The SAE advantage becomes smaller in later layers.

Can SAE Features Control Model Behavior?

Consider the IOI example:

"Then, Alice and Bob went to the store.

Alice gave a snack to ___."

The model should predict

Bob.

Now consider a modified version:

"Then, Alice and Vanessa went to the store.

Alice gave a snack to ___."

Can SAE Features Control Model Behavior?

For a selected feature f_j,

$$c_j = \text{its activation in the original sentence,}$$

$$\bar{c}_j = \text{its activation in the modified sentence.}$$

Then patch the model by replacing c_j with $\bar{c}_j$ for some selected features f_j.

Question

How many feature activations must we change before the model behaves as if it is processing the modified sentence?

Result: SAE Features Localize the Behavior Better

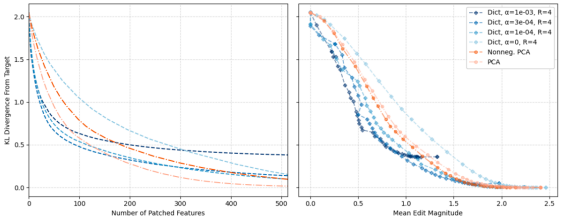
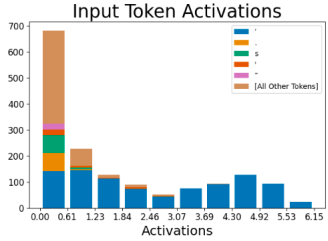
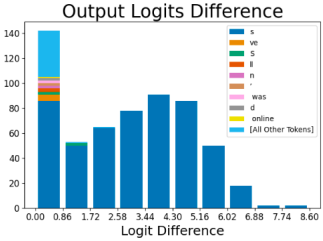


Figure 3: (Left) Number of features patched vs KL divergence from target, using various residual stream decompositions. We find that patching a relatively small number of dictionary features is more effective than patching PCA components and features from the non-sparse $\alpha = 0$ dictionary. (Right) Mean edit magnitude vs KL divergence from target as we increase the number of patched features. We find that our sparse dictionaries improve the Pareto frontier of edit magnitude vs thoroughness of editing. In both figures, the feature dictionaries were trained on the first 10,000 elements of the Pile (Gao et al., 2020) (approximately 7 million activations) using the indicated α and R values, on layer 11 of Pythia-410M (see Appendix F for results on other layers).

- SAE reaches low KL using **fewer patched features** than PCA.
- It also does so with **smaller edits**.

Case Study: Feature 556 and Apostrophes





Evidence from the input

Among the inputs that activate feature 556, the majority of high activations occur on apostrophes;

Evidence from the output

Ablate feature 556. The output logit that decreases the most is “`s`”.
That makes sense if the feature detects an apostrophe in contexts such as “`'s`”.

The Feature Does Not Activate on Every Apostrophe

The paper finds several different apostrophe-related features:

`'s`, `'ll`, `n't`, ...

Does that refute monosemanticity?

No.

Monosemanticity means roughly:

one feature corresponds to one coherent pattern.

It does *not* mean that every category humans name with one symbol must correspond to exactly one feature.

Circuits

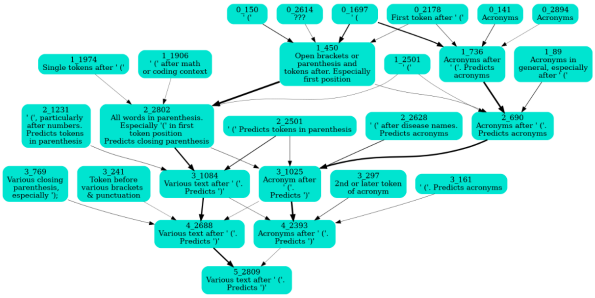


Figure 5: Circuit for the closing parenthesis dictionary feature, with human interpretations of each feature shown. Edge thickness indicates the strength of the causal effect between dictionary features in successive residual stream layers, as measured by ablations. Many dictionary features across layers correspond to similar real-world features and often point in similar directions in activation space, as measured by cosine similarity.

Upstream features detect situations that can lead to a closing parenthesis, such as:

- an opening parenthesis,
- dates,
- acronyms.

My Further Thoughts on “Correct” Interpretation

Perhaps there is simply no uniquely correct interpretation to be found.

At the implementation level, what happens in an LLM is ultimately a sequence of operations such as

$$\text{matrix multiplication} + \text{attention} + \text{nonlinear transformations} + \cdots$$

We introduce descriptions such as

“apostrophe feature”, “Texas feature”, “deception circuit”

because they make the model’s behavior easier for us to **understand, predict, and control**.

An Analogy with Geometry in Physics

General relativity represents gravitation using curved spacetime. But there are also equivalent formulations with a different mathematical structure: for example, representing gravity with torsion rather than curvature.

So what is the “correct” geometry of physical space?

Einstein famously argued in *Geometry and Experience* that we are not to ask what the “true” geometry of space is, but which geometry gives us the simplest and most useful physical theory.

Likewise for interpretability

We do not ask

“Is this the uniquely correct interpretation?”

but rather

Does this interpretation help us predict and control the model’s behavior?

References

Main reading

- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023.

Background

- Albert Einstein. *Geometry and Experience* (*Geometrie und Erfahrung*). 1921. English translation in *Sidelights on Relativity*, 1922.